OOP Basics

Overview

OOP stands for **Object-oriented programming.**

OOP is a style of programming where we build software using **objects** instead of just functions and data scattered around.

An **object** is something meaningful in your problem.

- For Example: user, bank account, order, employee, product, ticket, car.

Each object:

- **knows** something → its data (like name, balance, price).
- **does** something → its actions/behavior (like deposit, login, pay, print).

Problem with Procedural Programming

- **PoP** is series of instructions written in a **logical order**.
- For complex program, logic is broken down into **smaller blocks** of statements called **functions**.
- Returned value of one function can be used within other calling function(s).

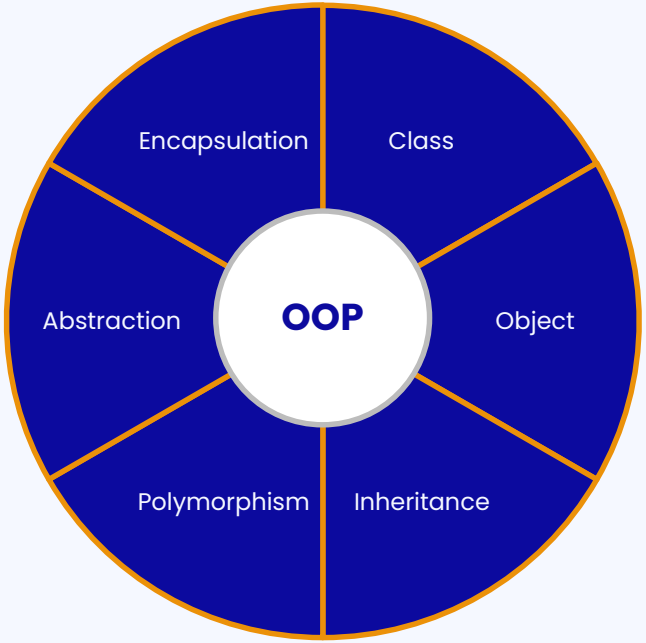
Problems:

- *Data and functions are **scattered***, so nothing clearly **"owns"** or protects important state.
- Heavy use of shared/global data *makes bugs **hard to track** and **easy to introduce***.
- **No clear boundaries or responsibilities**, so any part of the code can change anything.
- **Code becomes difficult to understand** and navigate as the project grows larger.
- Reusing and extending behavior often leads to **copy-paste code** and **inconsistencies**.

Why OOP?

- **Better Structure** – Groups related data and behavior into clear, self-contained units (objects).
- **Code Reuse** – Encourages reuse via classes and relationships instead of copy-paste.
- **Easy Maintenance** – Hides internals behind stable interfaces, so changes are safer.
- **Extensibility** – Allows adding new features/types without modifying existing core logic.
- **Real-World Modeling** – Represents real entities directly as objects, making designs intuitive.
- **Improved Testability** – Breaks systems into small, focused objects that are easier to test.

Characteristic of OOP



Class

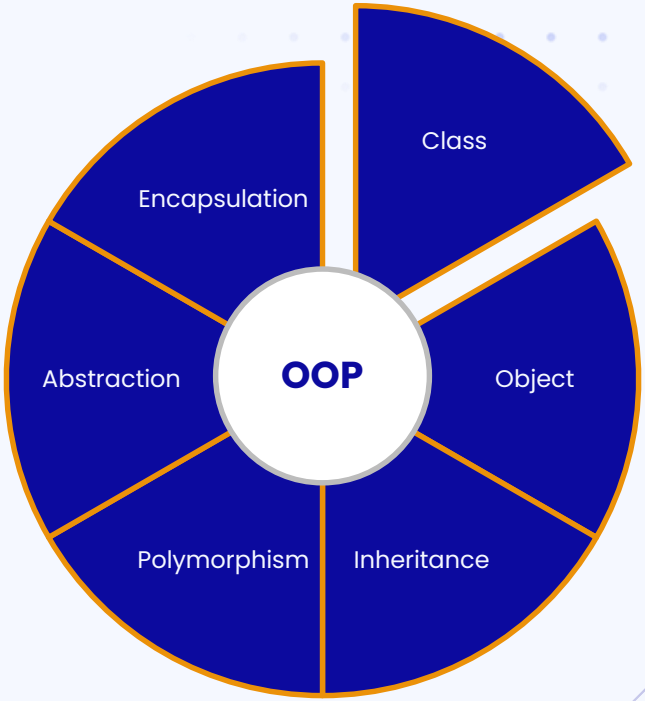
Definition:
A class is a blueprint or template that defines what a certain kind of object should know and do.

Structure:
It specifies the set of attributes (data) that its objects will hold.

Behavior:
It defines the operations/actions (methods) that its objects can perform.

Instance Creation:
Multiple individual objects (instances) are created based on one class.

Abstraction of a Concept :
It represents a general concept (e.g., “Student”, “Account”), not one specific example.



Object

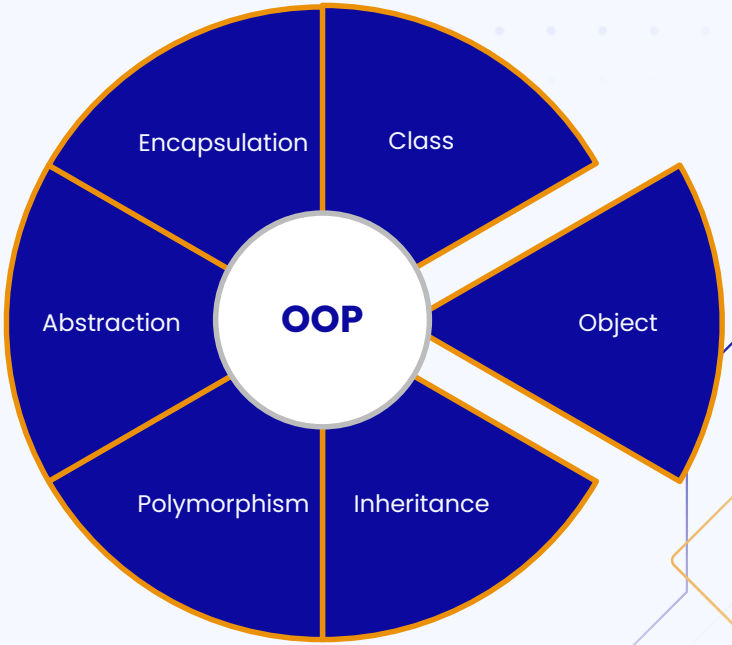
Definition:
An Object is an instance of a Class. It bundles related data and behavior together.

Instance of a Class:
It is a concrete example created from a class/blueprint.

State:
It stores its own data/values (like model, price, color), which can differ from other objects.

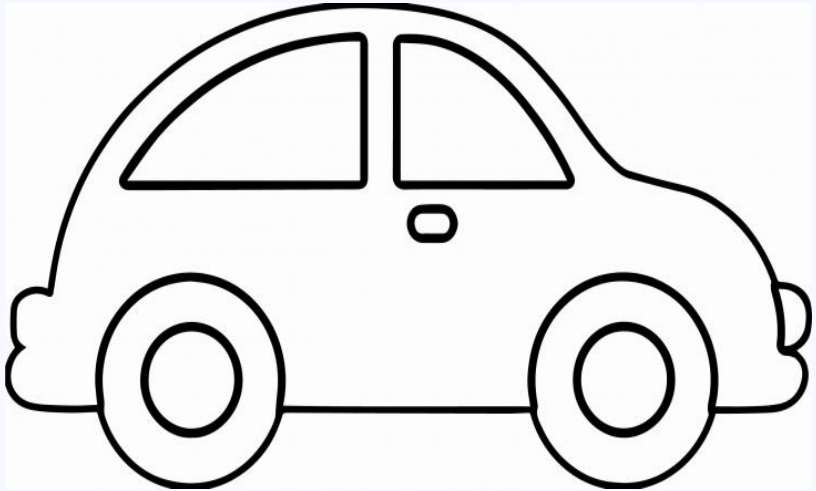
Behavior:
It can perform actions defined for its type (like accelerate, decelerate, break).

Identity:
Each object exists as a distinct unit, even if its data matches another object.



What is instance?

Instance = actual example or actual piece from a blueprint



Class (Generic Design / Blueprint)

Instances of Class (Real Specific Cars)

Inheritance

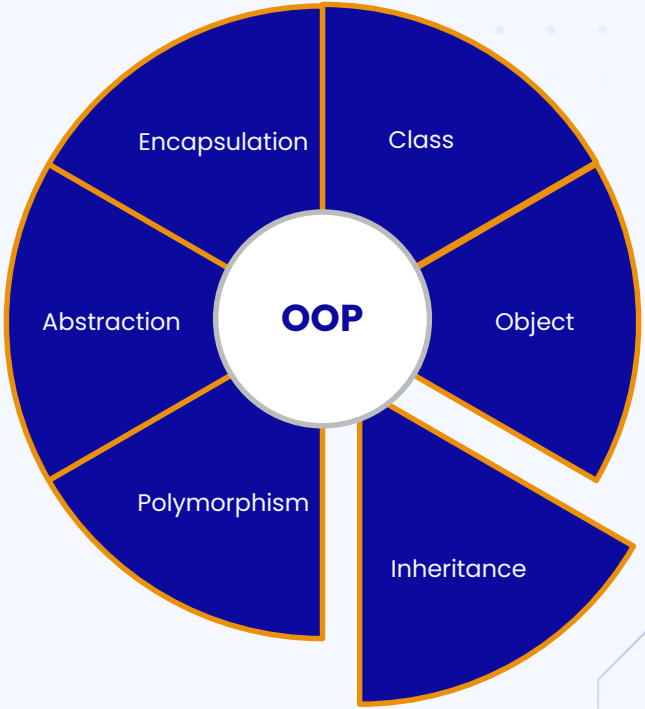
Definition:
Inheritance allows extending capability of an existing class (parent class) to build new class (child class)

Reuse:
Common attributes and behaviors are defined once in the parent and automatically available in the child.

Specialization:
Child classes can add new features or modify existing behavior to become more specific.

"Is-a" Relationship:
Inheritance models "X is a kind of Y" (e.g., Car is a kind of Automobile).

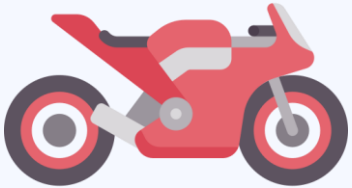
Less Duplication:
Reduces repeated code and keeps shared logic in one central place.



Inheritance Example

AUTOMOBILE

Car



Bike

Bus

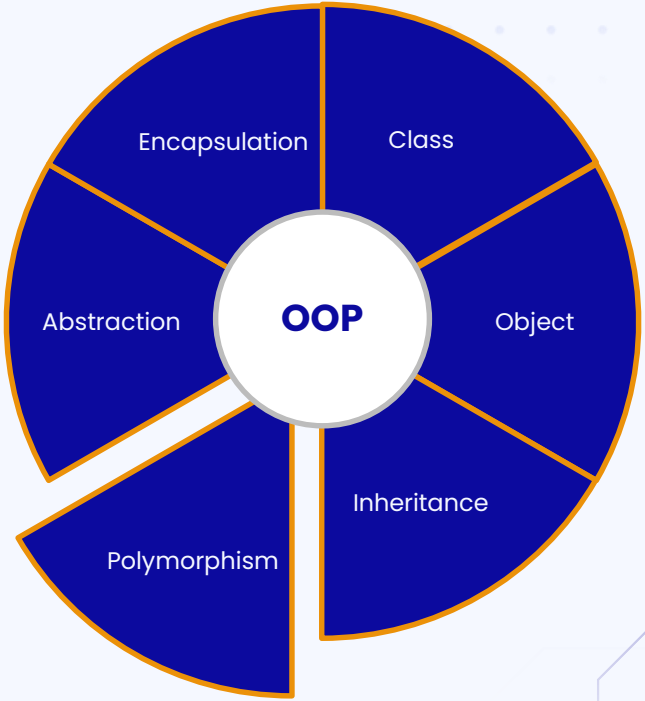
Polymorphism

Definition:
Polymorphism means the same operation or method name can work differently on different types of objects.

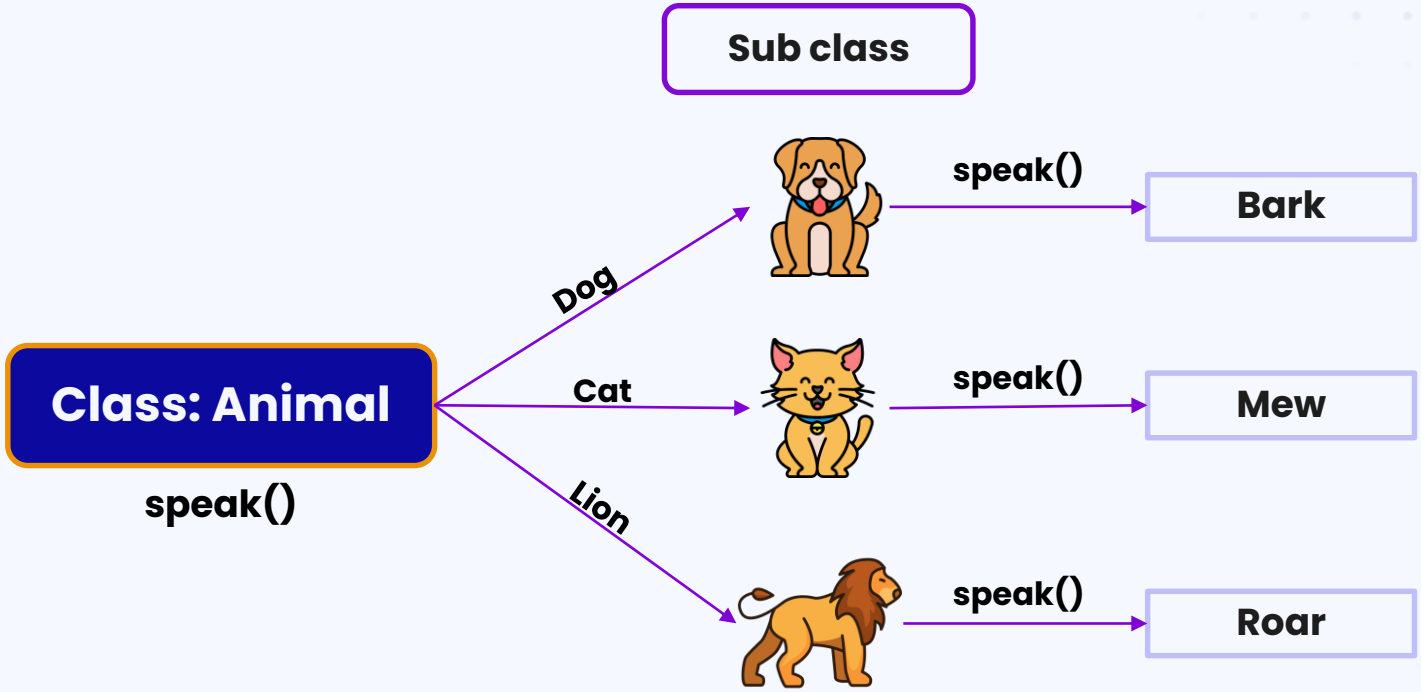
Common Interface:
Different classes provide the same method so they can be used interchangeably.

Different Behavior:
Each object gives its own implementation, but the calling code uses the same method call.

Supports Extensible Design:
Makes it easy to add new behaviors without modifying existing logic.



Polymorphism Example



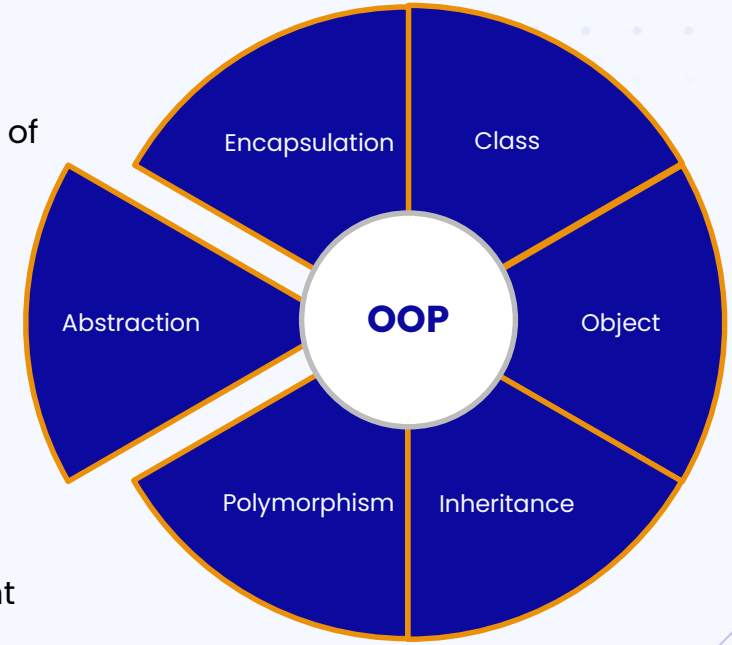
Abstraction

Definition:
Abstraction focuses on showing only the essential features of an object and hiding unnecessary details.

Reduced Complexity:
Helps users work with complex systems through simpler, high-level actions.

Implementation Hiding:
Internal logic can change without affecting code that depends on the exposed interface.

Foundation for Polymorphism:
Shared abstract behavior (common interface) lets different objects be used interchangeably.



Abstraction Example

We drive the car, not its engine internals.

Simple Controls Outside

Complex Logic Inside

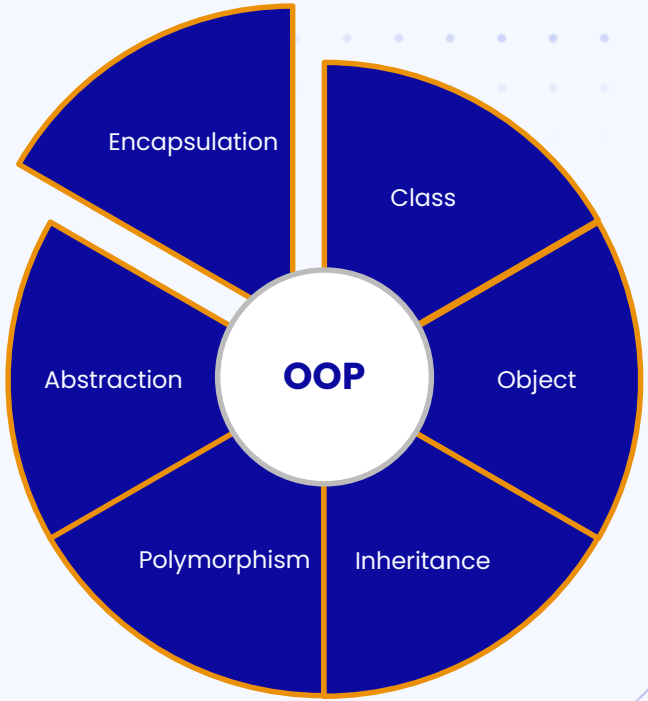
Encapsulation

Definition:
Encapsulation bundles data and the methods that operate on that data into a single unit (object).

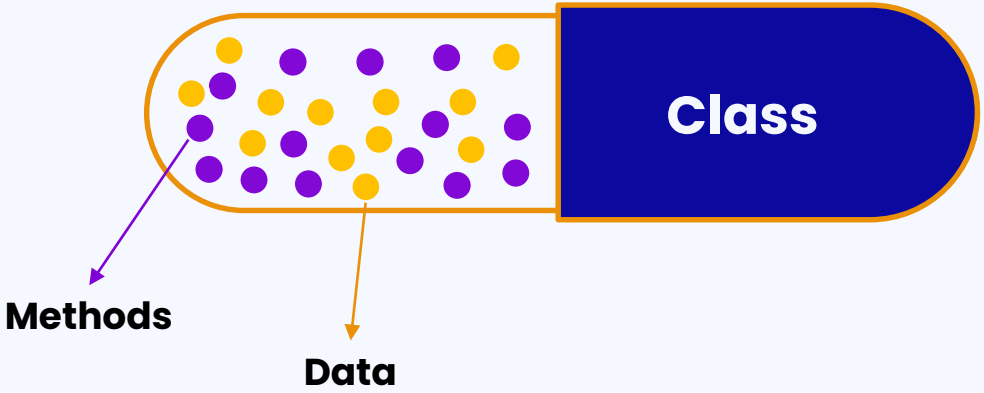
Controlled Access:
Internal details are accessed or changed only through well-defined methods, not directly.

Protection:
Prevents accidental or unauthorized modification of an object's internal state.

Change-Friendly:
Internal implementation can change without affecting code that uses the object's public interface.



Encapsulation Example



WATCH

Level up your coding with each episode in this focused Python series.

Next Video!

Class & Object